Assignment 1 Part 2: Architecture & Summary

1. Project Overview & Scope

Assignment 1 Part 2 (a12) builds upon the basics of Vulkan to implement a complete, robust rendering engine. The primary goal is to move beyond simple triangle drawing to a system capable of rendering complex 3D assets with advanced lighting and post-processing.

Key Features:

- **Physically Based Rendering (PBR)**: A modern lighting model that adheres to energy conservation principles, using Albedo, Roughness, and Metalness maps.
- **Dynamic Rendering**: Utilizes Vulkan 1.3's `vkCmdBeginRendering` to manage render passes without the boilerplate of `VkRenderPass` and `VkFramebuffer`.
- **Multi-Pass Architecture**: Implements an offscreen High Dynamic Range (HDR) pass followed by a Tone Mapping resolve pass.
- **Debug Visualization**: Runtime switching between standard rendering and debug modes (Mipmaps, Depth, Derivatives).

---

2. System Architecture

The application is structured around a central "Game Loop" in `main.cpp`, supported by a library of helper functions (`labut2`) and specific modules (`setup.cpp`, `rendering.cpp`).

2.1 The Application Lifecyle (`main.cpp`)

1. **Bootstrap**:
    - Initialize `volk` (Vulkan loader).
    - Initialize GLFW (Windowing).
    - **Device Selection**: We score available GPUs. Discrete GPUs (NVIDIA/AMD) are preferred over Integrated ones. We strictly require Vulkan 1.3 support.
    - **Queue Creation**: We retrieve a Graphic Queue and a Presentation Queue. Ideally, these are the same to avoid synchronization overhead.
2. **Resource Allocation**:
    - **VMA (Vulkan Memory Allocator)**: Initialized to handle all GPU memory allocations.
    - **Command Pool**: Created with `RESET_COMMAND_BUFFER_BIT` to allow per-frame reusing of command buffers.
    - **Swapchain**: Created with `VK_FORMAT_B8G8R8A8_SRGB` for automatic gamma correction.
3. **Pipeline Creation** (via `setup.cpp`):
    - **Layouts**: Define resources (Uniforms, Textures).
    - **Pipelines**: Pre-compile shader state (Blending, Rasterizer, Depth Test). We create:
        - `pipe`: Opaque geometry (No blending).
        - `alphaPipe`: Transparent geometry (Alpha blending enabled).

    - `*Pipe` (Debug): Special shaders for visualization.
    - `postProcPipe`: Full-screen quad shader.

4. **The Render Loop**:
    - **Sync**: `vkWaitForFences` (CPU wait) -> `vkAcquireNextImageKHR` (Swapchain wait).
    - **Update**: `update_user_state` (Input) -> `update_scene_uniforms` (Matrices).
    - **Record**: `record_commands` (The heavy lifting).
    - **Submit**: `vkQueueSubmit2` (Execute on GPU).
    - **Present**: `vkQueuePresentKHR` (Show on screen).

---

3. Data Flow & Resource Management

3.1 Vertex Data Flow

1. **Disk**: `mesh.positions` (std::vector).
2. **CPU RAM**: `vmaMapMemory` -> Copy to **Staging Buffer** (Host Visible).
3. **GPU Transfer**: `vkCmdCopyBuffer` -> Copy to **Device Local Buffer** (VRAM).
4. **Shader**: `vkCmdBindVertexBuffers` -> Vertex Shader Input `location = 0`.

3.2 Texture Data Flow

1. **Disk**: Read image file (stb_image).
2. **Staging**: Upload pixels to Staging Buffer.
3. **Transition**: `VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL`.
4. **Transfer**: `vkCmdCopyBufferToImage`.
5. **Mipmaps**: `vkCmdBlitImage` (downsample 0->1, 1->2...).
6. **Shader**: `vkCmdBindDescriptorSets` -> Fragment Shader Sampler.

3.3 Uniform Data Flow

1. **Update**: CPU calculates `View * Projection`.
2. **Upload**: Update **Staging Buffer** or Mapped Memory.
3. **Barrier**: `buffer_barrier` ensures upload finishes before draw.
4. **Shader**: Vertex Shader reads `uScene.projCam`.

---

4. The Render Graph (Detailed Control Flow)

Since we use Dynamic Rendering, the "Graph" is defined by the sequence of commands in `record_commands` (`rendering.cpp`).

Pass 0: Data Update

- **Barrier**: Ensure `SceneUBO` transfer is complete.

Pass 1: Scene Render (HDR)

- **Target**: `offscreenImage` (`R16G16B16A16_SFLOAT`).

- **Depth**: `depthBuffer` (`D32_SFLOAT`).
- **Clear**: Color (Black), Depth (1.0).
- **Action**:
    1. **Transition** Offscreen to `COLOR_ATTACHMENT_OPTIMAL`.
    2. `vkCmdBeginRendering`: Bind Offscreen+Depth.
    3. **Draw Opaque**: Bind `pipe`. Loop through meshes. Draw if `alphaMask` is null.
    4. **Draw Transparent**: Bind `alphaPipe`. Loop through meshes. Draw if `alphaMask` exists. *Note: Proper transparency requires sorting, which we approximate by drawing second.*
    5. `vkCmdEndRendering`.

Pass 2: Post-Processing Resolve (LDR)

- **Target**: `swapchainImage` (`B8G8R8A8_SRGB`).
- **Input**: `offscreenImage` (Sampled as Texture).
- **Action**:
    1. **Transition** Offscreen to `SHADER_READ_ONLY_OPTIMAL` (So we can sample it).
    2. **Transition** Swapchain to `COLOR_ATTACHMENT_OPTIMAL` (So we can write to it).
    3. `vkCmdBeginRendering`: Bind Swapchain.
    4. **Draw**: Bind `postProcPipe` + `postProcDescriptor` (pointing to Offscreen). Draw 3 vertices (Full-screen triangle generated in Vertex Shader).
    5. `vkCmdEndRendering`.

Pass 3: Presentation

- **Transition** Swapchain to `PRESENT_SRC_KHR`.
- Hand off to Presentation Engine.

---

5. Key Implementation Details

Double Buffering

We use `frameDone[i]`, `imageAvailable[i]`, and `renderFinished[i]` arrays.

- `i` cycles (0, 1, 0, 1...).
- Allows CPU to record Frame 2 while GPU executes Frame 1.
- Prevents "stalling" the pipeline, effectively doubling potential framerate.

Error Handling

We use a custom exception class `lut::Error`.

- All Vulkan calls are wrapped in `if( res != VK_SUCCESS ) throw ...`.
- This ensures we never silently fail (e.g., if a texture fails to load, the app performs a clean exit with a descriptive error).

Debug Modes

Implemented via shader permutations.

- **Standard**: Samples Main Texture.
- **Mipmap**: Shader visualizes `textureQueryLod`.
- **Depth**: Shader outputs `gl_FragCoord.z` (linearized).
- **Derivatives**: Shader outputs `dFdx(depth)` and `dFdy(depth)`.

This architecture ensures high performance (via minimal synchronization overhead and optimal memory usage) while maintaining maintainability through modular design.

---

6. API & Function Reference

6.1 Pipeline Setup (`setup.cpp`)

These functions encapsulate the verbose creation of Vulkan objects.

- **`create_triangle_pipeline_layout`**: Creates the pipeline layout for the main scene pass.
  - *Layout*: Set 0 (Scene UBO), Set 1 (Object Textures).
- **`create_post_proc_pipeline_layout`**: Creates the pipeline layout for the post-processing pass.
  - *Layout*: Set 0 (Offscreen Texture).
- **`create_triangle_pipeline`**: Creates the standard Opaque PBR pipeline.
  - *Features*: Backface culling, Depth Write/Test, No Blending.
- **`create_alpha_pipeline`**: Creates the Transparent pipeline.
  - *Features*: No culling (optional), Depth Write/Test, Alpha Blending (`SrcAlpha`, `OneMinusSrcAlpha`).
- **`create_debug_pipeline`**: Creates pipelines for debug views. Validates specific shader permutations (Mipmap, Depth, etc.).
- **`create_post_proc_pipeline`**: Creates the full-screen effect pipeline.
  - *Features*: No Depth Test, No Culling. Writes to Swapchain.
- **`create_depth_buffer`**: Allocates a GPU-only image for depth testing (`D32_SFLOAT`).
- **`create_offscreen_buffer`**: Allocates a GPU-only image for HDR rendering (`R16G16B16A16_SFLOAT`).
- **`create_debug_sampler`**: Creates a sampler with Anisotropy disabled (to visualize raw mip levels).

6.2 Rendering Logic (`rendering.cpp`)

- **`record_commands`**: The core function that records the entire frame's GPU commands into a command buffer.
  - *Args*: Pipelines, Descriptors, Buffers, Draw Data.
  - *Flow*: Update UBO -> Barrier -> Draw Scene -> Barrier -> Draw PostProc.
- **`submit_commands`**: Submits the recorded command buffer to the `GraphicsQueue`.
  - *Sync*: Waits on `imageAvailable` semaphore, Signals `renderFinished` semaphore.
- **`present_results`**: Queues the final image for presentation to the windowing system.

6.3 Camera & State (`camera.cpp`)

- **`update_user_state`**: Processes Input (Keyboard/Mouse) to update the Camera's position and orientation.
- **`update_scene_uniforms`**: Computes the View and Projection matrices based on the Camera state and uploads them to the `SceneUniform` struct.

6.4 Model Loading (`baked_model.cpp`)

- **`load_baked_model`**: deserializes the custom binary model format.
  - *Returns*: `BakedModel` struct containing vectors of `BakedTextureInfo`, `BakedMaterialInfo`, and `BakedMeshData`.
  - *Logic*: Reads magic header, parses texture paths, materials (indices), and mesh attributes (pos, norm, uv, index) into CPU-side vectors.

6.5 LabUtils (`labut2/`)

These are reusable wrapper classes provided to simplify Vulkan's verbosity.

- **`vulkan_window.hpp`**: Handles `volk` init, GLFW window creation, and Surface/Swapchain management.
- **`vkobject.hpp`**: A RAII wrapper (Move-only) for Vulkan handles to ensure `vkDestroy*` is called automatically.
- **`vkbuffer.hpp`** / **`vkimage.hpp`**: Wrappers combining the native Vulkan handle with the VMA allocation handle.
- **`descriptors.hpp`**: Helpers for `DescriptorSetLayout` creation.
- **`synch.hpp`**: Helpers for creating Fences/Semaphores and issuing Pipeline Barriers.